JavaScript Debugging Errors

Back to JS page

Table of Contents

Common Errors Explained

JavaScript error messages look scary, but most of them mean very simple things. This page explains the most common errors in beginner-friendly language.

Error messages are clues. They tell you where to look and what kind of mistake happened.

How to Read an Error Message

An error message usually has three important parts:

- The error type.
- A short explanation.
- A line number.

Click the line number in the console to jump to the exact line of code.

ReferenceError

A **ReferenceError** means a variable or function name does not exist. This is often caused by a misspelling or a missing declaration.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Errors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>ReferenceError</h4>
<p>Open the browser console (F12) to see the error. The variable is not declared.</p>
<p id="demo"></p>

<script>
try {
  console.log(myValue);  // ReferenceError: myValue is not defined
} catch (e) {
  document.getElementById("demo").innerHTML =
    "Error: " + e.name + " - " + e.message;
  console.log(e.name + ": " + e.message);
}
</script>

</body>
</html>
```

Example 1

Result View Example

TypeError

A **TypeError** means you used a value in an invalid way. This usually happens with `undefined` or `null`.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Errors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>TypeError</h4>
<p>Open the browser console (F12) to see the error.</p>
<p id="demo"></p>

<script>
try {
  let x;
  console.log(x.length);  // TypeError: Cannot read properties of undefined
} catch (e) {
  document.getElementById("demo").innerHTML =
    "Error: " + e.name + " - " + e.message;
  console.log("Tip: The variable exists, but it has no value. Log the value before using it.");
}
</script>

</body>
</html>
```

Example 2

Result View Example

SyntaxError

A **SyntaxError** means JavaScript cannot understand your code. This is often caused by missing brackets or parentheses. Syntax errors stop the script from running.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Errors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>SyntaxError</h4>
<p>Open the browser console (F12) to see the error from the eval below.</p>
<p id="demo"></p>

<script>
try {
  eval("if (x == 5 { console.log('Hello'); }");  // Missing closing parenthesis
} catch (e) {
  document.getElementById("demo").innerHTML =
    "Error: " + e.name + " - " + e.message;
  console.log("Tip: Check for missing brackets or parentheses.");
}
</script>

</body>
</html>
```

Example 3

Result View Example

NaN Errors

NaN means **Not a Number**. This happens when JavaScript cannot perform valid math.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Errors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>NaN Errors</h4>
<p>Open the browser console (F12) to see the result.</p>
<p id="demo"></p>

<script>
let result = "abc" * 5;
console.log("Result of 'abc' * 5:", result);  // NaN

if (isNaN(result)) {
  document.getElementById("demo").innerHTML =
    "Error: Result is NaN (Not a Number).<br>Tip: Check that both values are numbers before doin
} else {
  document.getElementById("demo").innerHTML = "Result: " + result;
}
</script>

</body>
</html>
```

Example 4

Result View Example

Cannot Read Property of Undefined

This is one of the most common beginner errors. It means you are trying to use something that does not exist. The variable exists, but the object does not.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Errors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Cannot Read Property of Undefined</h4>
<p>Open the browser console (F12) to see the error.</p>
<p id="demo"></p>

<script>
try {
  let user;
  console.log(user.name);  // Cannot read properties of undefined
} catch (e) {
  document.getElementById("demo").innerHTML =
    "Error: " + e.name + " - " + e.message;
  console.log("Tip: The variable 'user' exists but is undefined. Initialize it before accessing
}
</script>

</body>
</html>
```

Example 5

Result View Example

Common Error Meanings

- **ReferenceError** means a name is not defined.
- **TypeError** means a value is used incorrectly.
- **SyntaxError** means broken code structure.
- **NaN** means invalid math.

Debugging Tip

Do not ignore errors. Fix the first error before moving on. One error often causes many others.

Document

Document in project

You can Download PDF file.

Reference

- W3Schools JavaScript Debugging Errors